ICT374 Lab 5:
Writing Multi-module C Programs, Building Projects with `make` Utility and Makefile; Reading and Writing Files, Creating New Files, Setting Access Permissions on Files, Get File Attributes; Duplicating File Descriptors, Standard Input and Output Redirections, Getting File Attributes, Changing Read and Write Offsets, Non-blocking Reads, Accessing Directory Entries.

Learning Objectives:

1. Learn to define the interface of a module with a header file and to implement the module in C.
2. Learn to create a project that consists of multi-modules.
3. Understand and be able to write new makefiles and to build projects with `make` utility..
4. Learn to read and write files using file related systems calls such as `open`, `read`, `write`, `lseek`, and `close`.
5. Learn to create new files with `open` and `creat` systems calls and to delete files with `unlink`.
6. Learn to set access permissions on new files with `open` or `creat` systems calls.
7. Learn how to get file attributes, such as access permissions, the file type, i-node number, file size, ownership and time stamps, using systems call `stat`, `fstat` and `lstat`.
8. Learn to duplicate file descriptors and understand how shell's standard input and standard output redirections are implemented.
9. Learn how to change file descriptor flags and file status flags using systems call `fcntl`.
10. Learn how to read directory entries using directory related functions such as `opendir`, `readdir`, `rewinddir` and `closedir`.
11. Understand the difference between blocking read and non-blocking read and be able to set a file to non-blocking read.

Required Reading:

- Lecture Notes for Topic 5
- Rute: ch 22.4
- The manual pages of the process and file related related system calls and functions

C Programming Exercises

Note, all exercises must be done using terminal commands, not GUI tools. Do not use IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also each exercise should be in a separate directory, which itself, is under a directory called `lab05`

1. **Building multi-module C program - module header**

So far all our C programs consist of just a single `.c` file. In the real world, a C program often consists of many modules. Each module usually consist of two parts: the module header (the `.h` file) and the module body (the `.c` file).

The module header provides the interface of the module to the outside world. It *declares* what will be available to other modules. They usually consist of a list of constants, global variables, and function prototypes that other modules may use. You should never place anything in a module header unless it is required by another module. If something, such as a variable, is needed by the module body but not by any other module, place it in the module body rather than in the module header. The module header must ***not*** contain any function body or executable statement. The following module header declares the interface to a stack of strings:

```
/*
 * file:    stack.h
 * author:  HX
 * aim:     implements stack of strings
 * date:    2006.09.06
 */

typedef struct node {
     char *str;
     struct node *next;
} Node;

typedef struct {
     Node *top;
     int number;
}  Stack;

// create and return an empty stack
//
Stack *create_stack();

// push the string "str" into the stack "stack"
//
void push_stack(Stack *stack, char *str);

// pop off the string at the top of the stack "stack"
// and return the string
//
char *pop_stack(Stack *stack);

// return the number of strings in the stack
//
int size_of_stack(Stack *stack);

// chech whether the stack is empty
// return 1 if empty and 0 if not
//
int empty_stack(Stack *stack);
```

Save the above code in a `.h` file such as `stack.h`. Note the above header file contains type definitions (`Node` and `Stack`) and a list of function prototypes but there is no function body!

With a `.c` file, the compiler not only checks the syntax, it also converts the source code into an object code containing mainly machine instructions, if the source code is free of

syntax errors. What should happen when you compile a header file? Would the compiler also generate object code? The answer is no, as the header file is not supposed to contain any executable statements, so no object code will be generated. All the compiler will do is check whether there is any syntax error.

To just compile a source code without attempting to create an executable program we use the option -c in gcc, eg:

```
%   gcc -c stack.h
```

Now compile the header file to detect any syntax error. Since this header file is free of any syntax error, I would like you to introduce some errors into the code and then compile it. Correct the syntax errors and compile it again until it is free of any syntax error.

Once you have finished compilation, use the command `ls -l` to check your current directory, you will find no additional file have been created as the result of the compilation.

2. **Building multi-module C program - module body**

The module body implements the functions declared in the module header. These functions can be called *public* using the analogy of Java and C++, meaning that these functions are visible in other modules. The module body usually includes the module header at the beginning. If something, such as a variable, is already declared in the module header, it should ***not*** be re-declared in the module body. The module body may contain additional constants, types, variables, and functions that are required to implement those public functions. The latter functions can be called *private* in the sense they are hidden inside the module body and are not meant to be directly called from other modules. However you must remember that C is not very good in separating public functions and private functions and it is possible that one can call a function defined in a module body even if it is not declared in the module header, although we should try not to do so!

The following code implements a stack that is declared in the stack header:

```
/* file:     stack.c
 * aim:      implement a stack
 * author:   HX
 * date:     2006.09.06
 */

#include <stdlib.h>
#include <stdio.h>
#include "stack.h"

Stack *create_stack()
{
     Stack *s;

     s = malloc(sizeof(Stack));

     if (s==NULL) {
         fprintf(stderr, "No more memory - quit\n");
         exit(1);
     }
```

```
     s->top = NULL;
     s->number = 0;
     return s;
}

void push_stack(Stack *stack, char *str)
{
     Node *newNode;

     newNode = malloc(sizeof(Node));
     if (newNode == NULL) {
          fprintf(stderr, "No more memory - quit!\n");
          exit(1);
     }

     newNode -> str = str;
     newNode->next = stack->top;
     stack->top = newNode;
     stack -> number += 1;
}

char *pop_stack(Stack *stack)
{
     Node *np;
     char *str;

     if (empty_stack(stack))
         return NULL;

     np = stack->top;
     stack->top = np->next;
     stack->number -= 1;

     str = np->str;
     free(np);
     return str;
}

int size_of_stack(Stack *stack)
{
    return stack->number;
}

int empty_stack(Stack *stack)
{
    return stack->number == 0;
}
```

Save the above code into a `.c` file. It is a good programming practice to use the same file name as its module header. If you named the module header `stack.h`, it would be good idea to name the module body as `stack.c`.

Now you need to compile the module body. As the module body is not a stand-alone program (there is no main function!), we are not yet ready to create an executable program. So we will use option -c to get gcc to do the compilation only - the compiler will now check the syntax of the source code and, if it is free of syntax error, the compiler will go on and create an object code:

```
%  gcc -c stack.c
```

Now by using the command `ls` to check your current directory, you will find a file named `stack.o` is created. This is the object code for your source code `stack.c`.

Note that usually you do not need to compile the module header such as `stack.h` separately. When you compile the module body `stack.c`, the pre-compiler first inserts the header files (including `stack.h`) into the module body before compilation starts. Therefore, the module header is compiled together with the module body. To verify this, introduce a syntax error in the header file `stack.h`, then compile module body `stack.c` to see if the compiler would pick up any syntax error in the header file.

3. **Using modules in your C programs**

We are now ready to use module `stack` developed in Exercise 1 and 2 above. We will write a test program that will push a sequence of strings into the stack, and then pop these strings off the stack and print them one by one. As the strings from the stack will come out in the First-In-Last-Out order, we will see the strings printed in the reverse order.

In order to save time to create a sequence of strings, our test program will simply get the strings from the command line, as shown in the following code:

```
/* file:      test_stack.c
 * aim:       to test stack module
 * author:    HX
 * date:      2006.09.06
 */

#include <stdlib.h>
#include <stdio.h>
#include "stack.h"

int main(int argc, char *argv[])
{
     Stack *s;
     int i;

     // create an empty stack
     s = create_stack();

     // push all command line arguments into the stack
     for (i=0; i<argc; ++i)
        push_stack(s, argv[i]);

     printf("there are %d strings in the stack\n", size_of_stack(s));

     // pop out the strings in the stack and print them out
     while ( !empty_stack(s) )
          printf("Stack item: %s\n", pop_stack(s));

     printf("now there are %d strings in the stack\n", size_of_stack(s));

    exit(0);
}
```

Save the above code into a file, such as `test_stack.c`. Then compile the source code:

```
%  gcc -c test_stack.c
```

This should generate an object code `test_stack.o` for the source code.

Finally we are ready to link all the object files together to form a complete, executable program:

```
%  gcc test_stack.o  stack.o  -o teststack
```

The above command links the two object files together with the necessary library functions to create an executable program named `teststack`.

Try to run the program with several command line arguments, such as

```
%   ./teststack   one two three four five
```

4. **Using Makefile to Build a Project**

In Exercise 3 above, the program consists of three source code files: `stack.h`, `stack.c` and `test_stack.c`. This is a small program! Most real-world applications consist of many dozens of modules and some consist of hundreds of modules. We call such a multi-module program a project. Imagine what you need to do if a few of the these modules in a big project are modified. All source code files that use the modified modules will have to be re-compiled, and then all object files will have to be re-linked with relevant libraries to create a new executable. Since the dependency relationship between these modules are often complex, to find out which modules need to be re-compiled and the order of compilation can be a daunting task. To do this manually is both time consuming and error-prone.

To automate the process of compilation and linking, professional programmers use the `make` utility. The `make` utility relies on dependency rules stored in a special file called *makefile*. The `make` utility reads the rules from the makefile and then examines the modification time stamps of relevant files to see whether any update is required and take the specified actions to update (such compile or link) the relevant files automatically.

Use the project from Exercise 3 as an example. We have the following dependency relationship:

- The executable program `teststack` is dependent on the object files `test_stack.o` and `stack.o`. If any one of the two object files is modified after the executable was created, the executable must be re-created.
- The object file `test_stack.o` is dependent on source code `test_stack.c` as well as header file `stack.h`, since `test_stack.c` includes `stack.h`. If any one of the two source code files is modified after the object code was generated, the object file must be re-created by re-compilation.
- The object file `stack.o` is dependent on the header file `stack.h` and source code `stack.c`. If any one of the source code files is modified after the object code was generated, the source code `stack.c` must be re-compiled to generate the object code again.

The above dependency relationship is expressed in the following makefile:

```
# makefile for teststack
# the filename must be either Makefile or makefile
```

```
teststack: test_stack.o stack.o
TABgcc test_stack.o stack.o -o teststack

test_stack.o: test_stack.c stack.h
TABgcc -c test_stack.c

stack.o: stack.c stack.h
TABgcc -c stack.c

clean:
TABrm *.o
```

The above makefile consists of a list of *dependency rules*. Each dependency rule specifies:

a. a *target* such as `teststack` in the line

```
teststack: test_stack.o stack.o
```

in the above makefile;

b. and which files the target is dependent on, such as file `test_stack.o` and `stack.o` in the line

```
teststack: test_stack.o stack.o
```

in the above makefile. The above line says that the target `teststack` is dependent on files `test_stack.o` and `stack.o`. By comparing the modification time stamps of the three files, if it finds that one of the `.o` files is modified after the file `teststack` was created, the action specified in the line below it must be taken.

c. and what *action* must be taken once the dependency is violated. For example, the line

```
TABgcc test_stack.o stack.o -o teststack
```

specifies the action of re-linking the object files `test_stack.o` and `stack.o` with relevant library routines to create a new executable program `teststack`.

Note each action line must be proceeded by a single TAB character and the TAB character must be the first character in that line. In the above makefile, you must replace the three letter "TAB" with a single TAB character.

The makefile given above defines four targets. The last target `clean` is unconditional - it is not dependent on anything.

To make a target, we use the command `make` with the target name. For example, to remove all object files, we use the command:

```
%   make clean
```

To make the first target, ie to create the executable `teststack`, we use

```
%  make teststack
```

or simply

```
% make
```

when the target is the first target, as is the case with target `teststack`.

Section 22.4 of Rute contains more details about `make` utility and *makefiles*.

Save the above makefile into a file named `Makefile`. Try to recompile program `teststack` for the following scenarios:

a. the file `stack.c` is modified. You can simulate the modification of a file by changing its modification stamp using command `touch` as in:

```
% touch stack.c
```

b. the file `test_stack.c` is modified.

c. the file `stack.h` is modified.

In each scenario listed above, observe what action or actions are taken by the `make` utility.

Unix Programming Exercises

1. **Reading from a text file**

   Create a text file with an editor. Then write a program to read that file in using *file systems calls* and print the content of the file to the standard output (for this part you may use printf).

2. **Copying files**

   Modify that program from Unix Programming Exercise 6 above, so that it copies the input file "foo" to the file "foo2". Make sure that you create the file "foo" and "foo2" before running the program. Check that the file has been correctly copied.

   For this question, you must use only file related system calls, not the standard I/O functions from C.

3. **Creating file if it does not exist**

   Modify the program from Unix Programming Exercise 2 above so that it works even if the file "foo2" doesn't exist.

4. **Setting correct permissions for new files**

   Modify the program in Unix Programming Exercise 3 above so that the new file "foo2" is readable and writable by everyone.

   What permission number would you use in your open/creat call? Check the actual permissions on the file. Are they the same as you specified in your program?

   Type the command `umask` at your shell prompt to find out the reason.

5. **Get file attributes**

   Write a program that gets a file name from its command line. Then it reports the following information about the file:

   - the type of file;
   - its access permissions;
   - its i-node number;
   - its size.

6. **Changing file offsets**

   Create a text file with an editor. Type in a few line of text. Then write a program that changes the last 20 characters of that file to "#". Compare the original file with the updated file.

7. **Listing directory entries**

   Write a program that gets a directory name from the command line. Then it lists all file names (and their i-node numbers) under the directory including the directory itself and its parent directory.

   Test your program on at least your home directory and the root directory "/".

8. **File descriptor 0 & 1 - what are they?**

   Write a program to write a line of text to file descriptor 1. Run the program. What happens?

   Change the program so that it reads a buffer of bytes from file descriptor 0 and writes it to file descriptor 1. What are we trying to do?

9. **File descriptors 1 & 2 - what's the difference?**

   Write a program that writes one line of text to file descriptor 1 and another line of text to file descriptor 2.

   Run the program. What happens?

   Now do an experiment. Assume the name of the program is "ex09". Compare the outputs of the following three commands (assume % is the shell prompt):

   ```
   %  ex09
   %  ex09 > foo1
   %  ex09 2> foo2
   ```

   Can you explain the difference?

10. **Duplicating file descriptors**

    Write a program that duplicates the file descriptor of an exiting open file onto file descriptor 1. Then write a few lines of text using `printf`.

    What happens and why?

11. **Standard output redirection**

    Write a program that executes the command "ls -lt". Make sure the output from the command goes to file "foo".

12. **Non-blocking reads**

    Write a program that does the following two things: firstly it sleeps for 5 seconds. Then it reads a line of text from the standard input and write the line to the standard output.

    This program would block forever unless you type a line of text on the keyboard.

    Can you modify the program so that it would write its input line from the standard input to the standard output if there is an input line from the keyboard. Otherwise it just reports "no input!" and terminates?

The Weekly Lab

Submit the following exercises:

1. C Programming Exercise 4.
2. Unix Programming Exercise 5.
3. Unix Programming Exercise 7.
4. Unix Programming Exercise 11.

This weekly lab must be submitted to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submitting Weekly Labs when preparing and submitting a weekly lab.

Dr Hong Xie
Unit Coordinator

Last update: Thursday, 28 August 2025

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming
Last Modified: Friday, 29-Aug-2025 07:16:24 AWST
Disclaimer & Copyright Notice © 2026 Murdoch University
This document is relevant for 2025 only